Ayudantía 1

SISTEMAS DISTRIBUIDOS

PROFESOR: NICOLAS HIDALGO

AYUDANTE: VICENTE DIAZ HORMAZÁBAL

SECCIÓN 3

Contacto

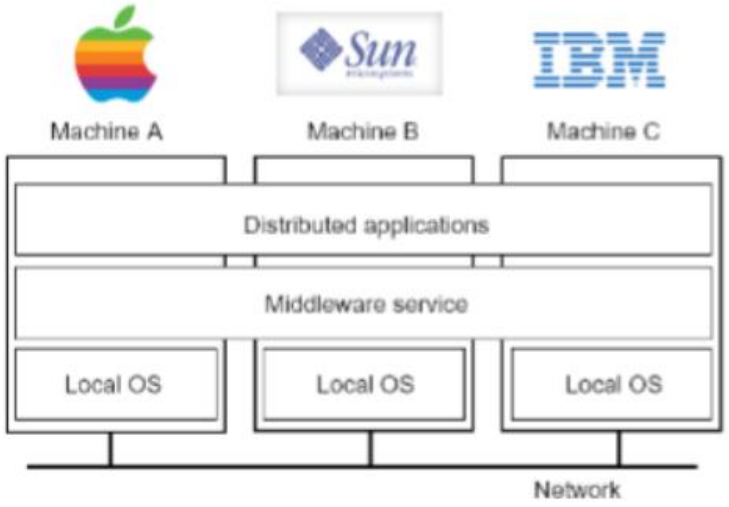
Discord: BarrosLucoPalta

WSP: +569 5395 5405

Steam: Coustillier

¿Qué son los sistemas distribuidos?

- Conjunto de máquinas que coordinan sus acciones y que aparecen al usuario como sistema único y coherente.
- Middleware: SD emplea software sobre la red para lograr cometido.
- Red de computadores: usuarios interactúan directamente con las máquinas.



¿Porqué y cuándo distribuir?

- Lograr escalabilidad, disponibilidad, eficiencia y cercanía al usuario.
- Supera limitaciones de un sistema centralizado.
- Dependencia al tamaño e infraestructura.

Ventajas:

- Compartir recursos.
- Mejora de desempeño.
- Mayor adaptabilidad.
- Mayor disponibilidad.
- Escalabilidad.

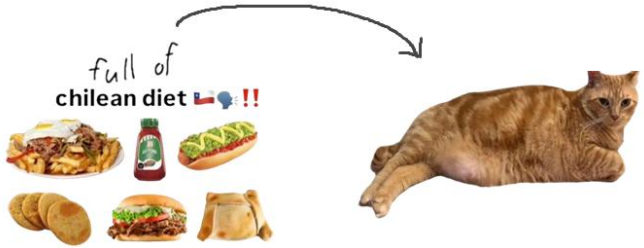
Desafíos:

- Heterogeneidad del hardware.
- Consistencia de la interacción.
- Siempre disponible.
- Usuarios y apps no deben notar caídas.
- Seguridad

Objetivos de diseño

Existen 4 objetivos al diseñar un sistema distribuido:

1) Poner recursos a disposición.
2) Hacer transparente la distribución.
3) Ser abierto.
4) Escalabilidad.



1) Poner recursos a disposición.

- Facilitar a los usuarios y aplicaciones acceder recursos remotos de manera eficiente.
- Razones para compartir:
  - Económicas.
  - Colaboración e intercambio de información.
- Si se incrementar conectividad y compartición de recursos la seguridad se vuelve relevante.
  - Seguridad de la comunicación.
  - Seguridad de acceso a los datos.

2) Hacer transparente la distribución.

- Esconder el hecho que procesos y recursos están físicamente distribuidos en múltiples computadores.
- Usuarios y aplicaciones ven un sistema único.
- Sistema coherente.

- Transparencia respecto a diferentes ámbitos:
  - Acceso.
  - Ubicación.
  - Migración.
  - Re-localización.
  - Replicación.
  - Concurrencia.
  - Falla.

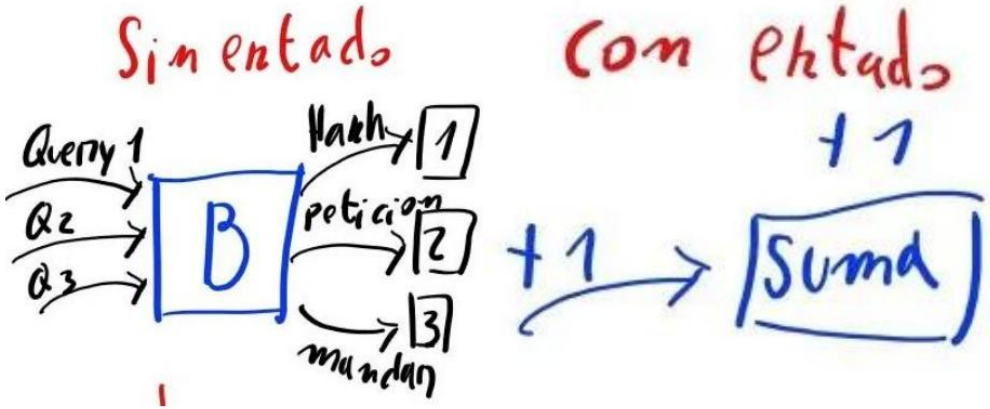
3) Ser abierto

- Sistema distribuido abierto es aquel que ofrece servicios bajo reglas estándar (sintaxis, semántica).
- Interface Definition Language (IDL), lenguaje de especificación de componentes del sistema, como, por ejemplo:
  - Advanced Programming Interface (API).
  - Llamadas a procedimientos remotos (RPC).
- Interoperabilidad: permite que procesos se comuniquen y generen implementaciones diferentes.
- Portabilidad: extensiones pueden ser usadas en otros sistemas
- Fácil extensión: agregar más componentes
  - En la práctica, muchos sistemas son monolíticos (cerrados).

4) Escalabilidad

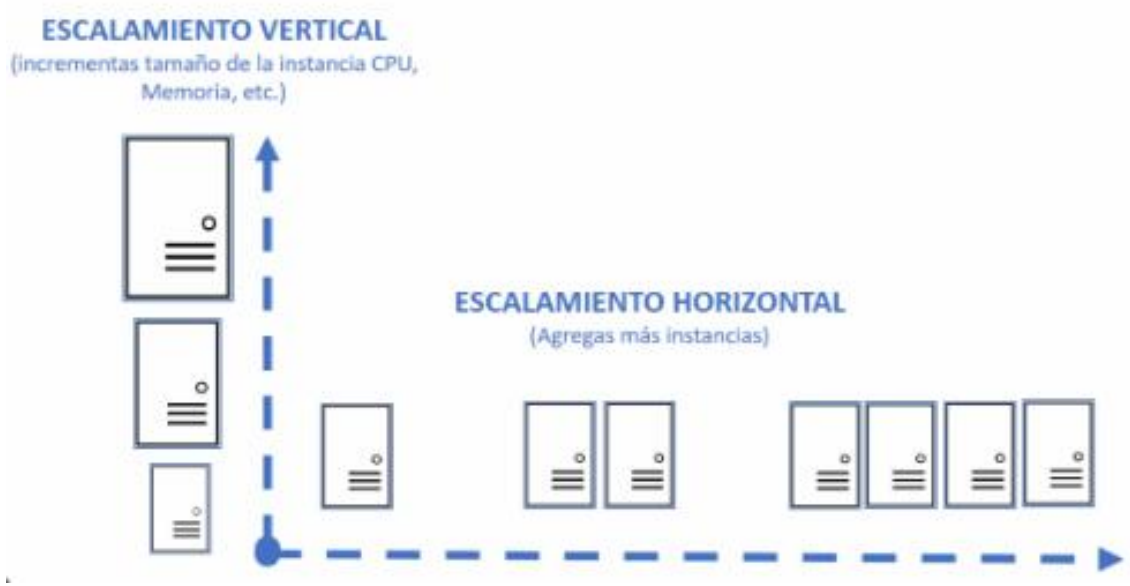
Existen 2 tipos:

- Escalabilidad Horizontal (scaling out): Aumentar la cantidad de los recursos.
  - Agregar más nodos de trabajo.
  - Front-End sin estado -> lo que haya pasado antes no influye en las próximas decisiones.
  - Backend con particionamiento, modelo master/slave.



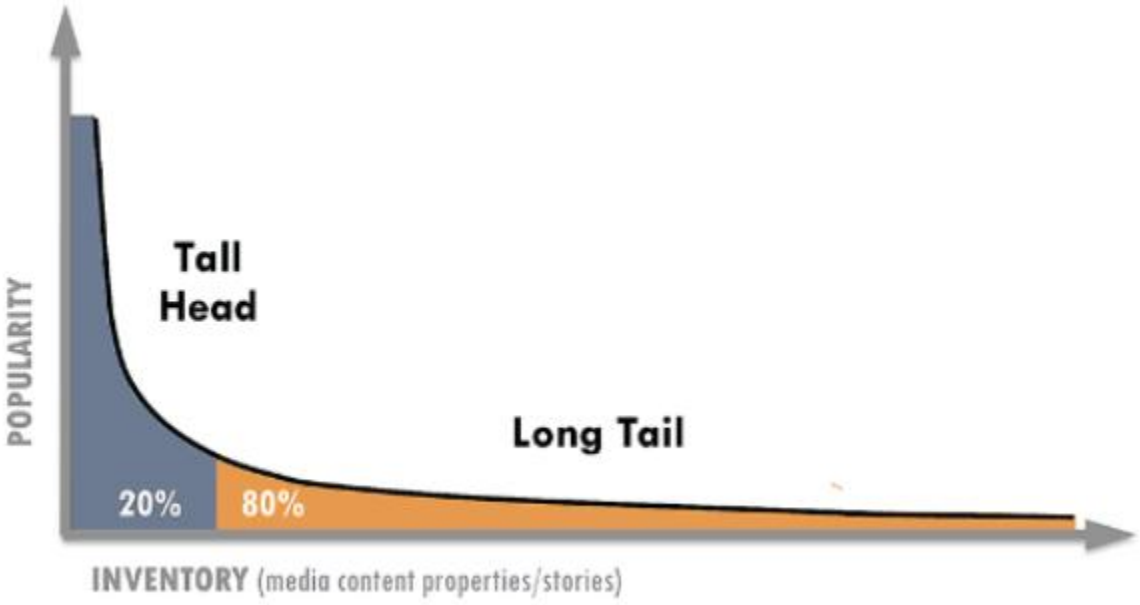
4) Escalabilidad

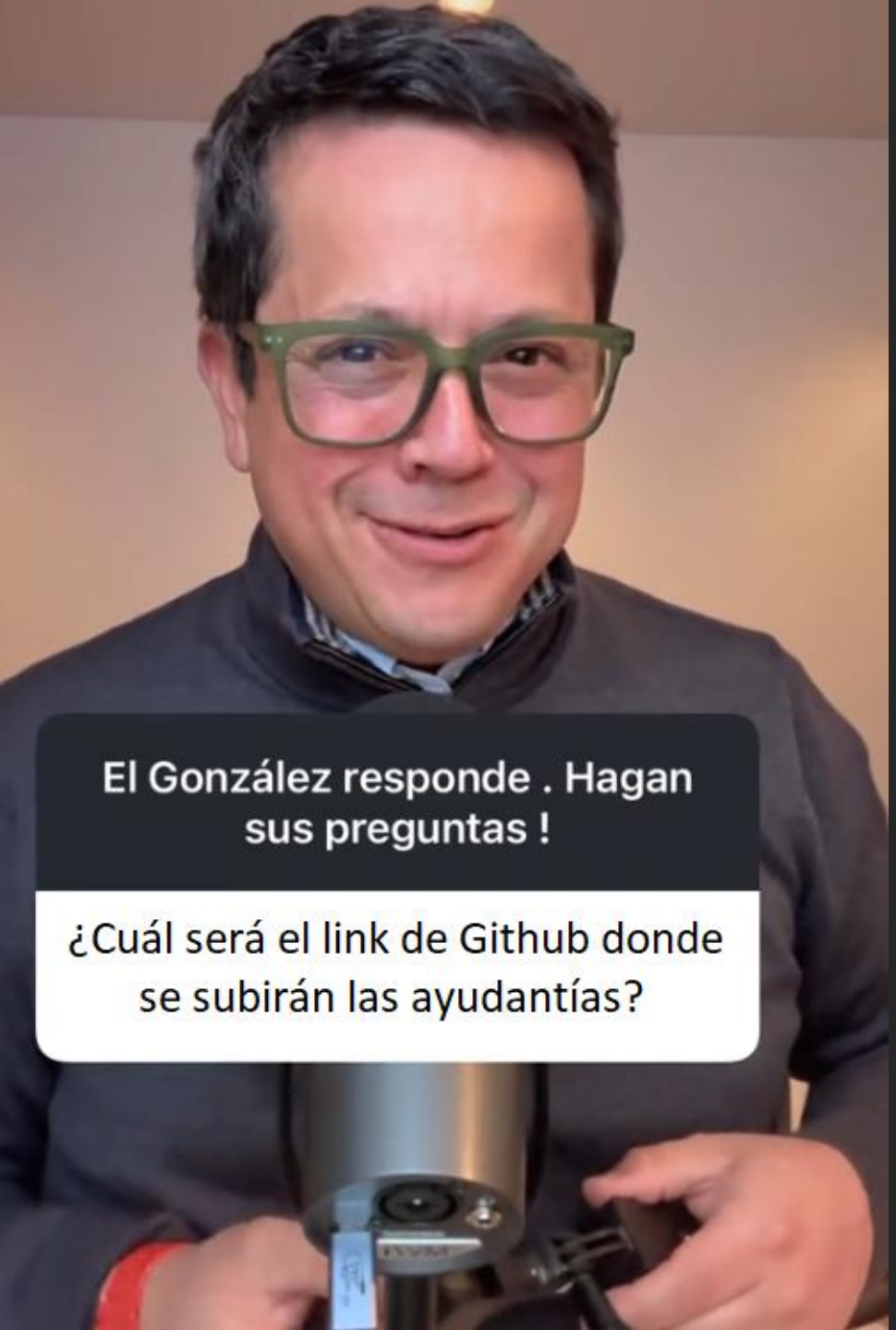
- Escalabilidad Vertical (scaling up): Aumentar las capacidades de mi recurso
  - Agregar CPU, Memoria, disco.
  - Manejar simultáneo de más conexiones, operaciones, usuarios.
  - Buen I/O y modelos de concurrencia (hebras, asincronismo, etc...).



4) Escalabilidad

- Existen 3 técnicas básicas para escalar:
  I. **Particionar y distribuir:** dividir un componente y distribuirlo, el cuál puede tener 2 variaciones:
     a. **Particionamiento Vertical**
     b. **Particionamiento Horizontal (Sharding)**
  II. **Replicar:** hacer copiar de los recursos en diferentes lugares.
  III. **Cache:** subconjunto de recursos de acceso fácil y rápido.





Link Github

Repositorio:

https://github.com/VicenteDiazH/AY-SD2026-1-S3